TASK-2

	Order_ID	Order_Date	Product	Category	Quantity	Unit_Price	Sales_Amount	Region
0	1001	2026-01-05	Laptop	Electronics	2	50000.0	100000	South
1	1002	2026-01-06	Smartphone	Electronics	3	20000.0	60000	North
2	1003	2026-01-07	Office Chair	Furniture	5	3000.0	15000	East
3	1004	2026-01-08	Headphones	Electronics	4	1500.0	6000	West
4	1005	2026-01-09	Desk	Furniture	2	7000.0	14000	South
5	1006	2026-01-10	Tablet	Electronics	NaN	18000.0	36000	North
6	1007	2026-01-11	Laptop	Electronics	1000	50000.0	50000000	East
7	1008	2026-01-12	Smartphone	electronics	2	20000.0	40000	south
8	1009	2026-01-13	Office Chair	Furniture	3	3000.0	9000	West
9	1009	2026-01-13	Office Chair	Furniture	3	3000.0	9000	West
10	1011	2026-01-15	Monitor	Electronics	2	NaN	24000	North
11	1012	2026-01-16	Keyboard	Electronics	5	800.0	3500	East
12	1013	2026-01-17	Printer	Electronics	-2	12000.0	-24000	West
13	1014	2026-13-40	Desk	Furniture	2	7000.0	14000	South
14	1015	2026-01-19	Mouse	Electronics	Three	500.0	1500	N0rth

```
df.isnull().sum()
```

	0
Order_ID	0
Order_Date	0
Product	0
Category	0
Quantity	1
Unit_Price	1
Sales_Amount	0
Region	0

dtype: int64

```
df.duplicated().sum()
```

```
np.int64(1)
```

```
df_duplicates = df[df['Order_ID'].duplicated()]
print(df_duplicates)
```

```
   Order_ID  Order_Date       Product   Category Quantity  Unit_Price  \
9      1009  2026-01-13  Office Chair  Furniture        3      3000.0

   Sales_Amount Region
9          9000   West
```

```
duplicate_rows = df[df.duplicated()]
print(duplicate_rows)
```

```
   Order_ID  Order_Date       Product   Category Quantity  Unit_Price  \
9      1009  2026-01-13  Office Chair  Furniture        3      3000.0

   Sales_Amount Region
9          9000   West
```

```
df = df.drop_duplicates()

df.shape
```

```
(14, 8)
```

```
highest_priced_product = df[df['Unit_Price'] == df['Unit_Price'].max()]
print("Highest Priced Product:\n ",highest_priced_product[['Product','Unit_Price']])
```

```
Highest Priced Product:
    Product  Unit_Price
0  Laptop     50000.0
6  Laptop     50000.0
```

```
revenue_by_region = df[df['Quantity_Flag']=="Valid"].groupby("Region")['Sales_Amount'].sum()
print(revenue_by_region)
```

```
Region
East     50018500
North       60000
South      154000
West        15000
Name: Sales_Amount, dtype: int64
```

```
valid_df.to_csv("valid_sales_dataset.csv",index=False)
```

```
valid_df
```

	Order_ID	Order_Date	Product	Category	Quantity	Unit_Price	Sales_Amount	Region	Quantity_Flag
0	1001	2026-01-05	Laptop	Electronics	2.0	50000.0	100000	South	Valid
1	1002	2026-01-06	Smartphone	Electronics	3.0	20000.0	60000	North	Valid
2	1003	2026-01-07	Office Chair	Furniture	5.0	3000.0	15000	East	Valid
3	1004	2026-01-08	Headphones	Electronics	4.0	1500.0	6000	West	Valid
4	1005	2026-01-09	Desk	Furniture	2.0	7000.0	14000	South	Valid
6	1007	2026-01-11	Laptop	Electronics	1000.0	50000.0	50000000	East	Valid
7	1008	2026-01-12	Smartphone	Electronics	2.0	20000.0	40000	South	Valid
8	1009	2026-01-13	Office Chair	Furniture	3.0	3000.0	9000	West	Valid
11	1012	2026-01-16	Keyboard	Electronics	5.0	800.0	3500	East	Valid

1. Missing values in columns

Cause:

- Data entry was incomplete.
- Information was not recorded during a transaction.
- Data was lost during import or collection.

Solution: Identify missing values and fill them when the correct value can be reliably calculated or verified. Otherwise, flag them for investigation.

2. Duplicate orders

Cause:

- The same transaction was entered twice.
- Data was imported more than once.
- Duplicate records were created during system synchronization.

Solution: Identify duplicate rows and remove exact duplicates after verification.

3. Find the highest-priced product

Cause of the analysis requirement:

- The business wants to identify its most expensive product.
- This helps with pricing analysis, product positioning, and inventory decisions.

Solution: Find the maximum Unit_Price and retrieve the corresponding product.

4. Group revenue by region

Cause of the analysis requirement:

- The business wants to compare sales performance across regions.
- Regional revenue analysis helps identify strong and weak markets.

Solution: Group sales amounts by region and calculate the total revenue. Exclude records with invalid or missing quantity flags.